System Programs and Operating-System Structures

Lecture Notes — Lecture 4

Monolithic, Layered, Microkernel, Modular, Virtual Machine

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

October 29, 2026

Concepts That Click

4.1 One Bad Driver

Imagine a network-card driver with a bug: on a rare packet, it dereferences a `NULL` pointer. Does the whole machine crash, does only the network stop, or does nobody notice? The answer is *not* in the driver's code. It is in how the operating system is **organized**. Where the driver runs — in kernel mode or user mode, inside the kernel or beside it — decides how far the damage spreads. This chapter drops that same buggy driver into five structures in turn and watches its *blast radius* change.

Lecture 3 studied the system-call doorway. This chapter studies what sits on each side of it. Above the doorway are the **system programs**, ordinary user-mode programs that make system calls. Below it is the kernel, whose internal organization — simple, monolithic, layered, microkernel, modular, or a guest of a virtual machine — is its **structure**. Between the two we need a small amount of design vocabulary: design goals, and the separation of mechanism from policy. The chapter closes with a single comparison table: one driver bug, five outcomes.

Reading: Silberschatz et al., Ch. 2 §2.5–2.8, pp. 66–84 [1]; Stallings, Ch. 2 and Ch. 4 §4.3 [2].

4.2 System Programs

Reading: Silberschatz et al., Ch. 2 §2.5, p. 66.

Most of what a user calls "the operating system" is not the kernel at all. It is a set of ordinary programs that make system calls.

Definition (System program). *A **system program** (system utility) is a program shipped with the operating system that gives users a convenient environment for program development and execution. It runs in user mode and obtains every privileged service through system calls.*

`ls`, `cp`, `cc`, `vi`, the shell, and the login program are all system programs. Some are thin wrappers around one system call; others are large programs in their own right. Either way, the user's view of the operating system is defined by the system programs, not by the system calls: a user types `ls`, not `getdents`.

Silberschatz groups system programs into six categories (Table 4.1). Beyond these, many systems also ship **application programs** — browsers, word processors, databases — which are not part of the operating system proper.

Example 4.1 (Classifying six commands)**.** *Classify each command by system-program category.*

Category	What it does	Examples
File management	create, delete, copy, rename, list files and directories	`cp`, `mv`, `rm`, `ls`
Status information	report date, time, memory, users, performance	`date`, `who`, `df`
File modification	create and change file contents	`vi`, `grep`, `sed`
Programming-language support	compilers, assemblers, interpreters, debuggers	`cc`, `as`, `gdb`
Program loading and execution	loaders, linkage editors	`ld`, the loader
Communications	connections between processes, users, and machines	`ssh`, `mail`, `ftp`

Table 4.1: The categories of system programs (Silberschatz et al., Ch. 2 §2.5, p. 66).

1. *`ls -l` lists a directory's entries, using `open`, `getdents`-style reads, and `stat`:* ***file management.***
2. *`date` reads the clock through an information-maintenance system call:* ***status information.***
3. *`vi notes.txt` edits contents with `read` and `write`:* ***file modification.***
4. *`cc prog.c` compiles source into an object file:* ***programming-language support.***
5. *`ld prog.o` links objects into an executable:* ***program loading and execution.***
6. *`ssh server` opens a connection with `socket`:* ***communications.***

Notice that there are two taxonomies in play. A system program *category (this chapter) is not the same thing as a system* call *category (Lecture 3). `date` is a status-information* program *that makes an information-maintenance* call.

4.2.1 The command interpreter

The **shell** reads a command and gets it executed. There are two ways to build it (Silberschatz et al., Ch. 2 §2.2). In the first, the interpreter contains the code for each command as a *built-in*; adding a command means changing the interpreter, and the interpreter grows with every command. In the second, used by UNIX, commands are *system programs*: `rm file.txt` makes the shell find a *file* named `rm`, load it, and pass `file.txt` as its argument. New commands are new files, and the shell stays small. The UNIX shell uses the second design for most commands, and keeps a few built-ins such as `cd`.

Why must `cd` be a built-in? A separate program would run in a *child* process created by `fork`. Calling `chdir` in that child changes only the child's current directory, and the child then exits. The shell's own directory would be unchanged, so the shell must call `chdir` itself.

4.3 Design Principles

Reading: Silberschatz et al., Ch. 2 §2.6.

Concepts That Click

Before choosing a structure, a designer decides what the system is for. Users want a system that is convenient, easy to learn, reliable, safe, and fast (**user goals**). The people who build it want one that is easy to design, implement, and maintain, and that is flexible, reliable, error-free, and efficient (**system goals**). These goals conflict, so there is no single "best" operating system — only a best fit to a set of requirements.

The second principle keeps the decisions about *what* to do apart from the machinery for *how* to do it.

Definition (Mechanism vs. policy). *A **mechanism** determines* how *to do something; a **policy** determines* what *will be done. Separating them lets a policy change without rewriting the mechanism.*

Example 4.2 (Separating mechanism from policy)**.** *Table 4.2 separates four situations into their mechanism and policy. The split pays off because the timer mechanism of Lecture 2 is unchanged*

Situation	Mechanism (how)	Policy (what)
CPU protection	the timer interrupts the CPU after a set interval	how long the interval is for each user
Scheduling	a context switch from one process to another	which process runs next
File protection	the kernel checks a permission bit before `open`	who is granted which bits
Our driver	loading code into the kernel at run time	which drivers are trusted enough to load

Table 4.2: Mechanism versus policy in four situations.

whether the quantum is 10 ms or 100 ms; only a parameter changes. A microkernel (Section 4.6) pushes this idea to its limit: the kernel keeps the mechanisms, and user-level servers hold the policy.

4.3.1 Implementation language

Early operating systems were written in **assembly language**. Modern kernels such as Linux and Windows are written mostly in **C**, with small parts in assembly: the trap entry, the context switch, and device-register access. A high-level language is faster to write, easier to read and debug, and easier to **port** to new hardware. The possible cost is slower code, but the bottlenecks are usually algorithms and data structures, which can be tuned after profiling. The structure of the code therefore matters more than its language: a well-structured C kernel can be optimized where it counts.

4.4 Simple and Monolithic Structure

Reading: Silberschatz et al., Ch. 2 §2.7, pp. 70–76.

A kernel is millions of lines. How those lines are divided into parts — and which parts share kernel mode — is its structure.

Concepts That Click

4.4.1 Simple structure: MS-DOS

MS-DOS was written for small machines, and its levels are not well separated (Figure 4.1). Applications can call BIOS routines and write to the display directly, because the Intel 8088 for which it was written had **no dual mode** and no hardware protection. The consequence is that one bad program can crash the whole system.

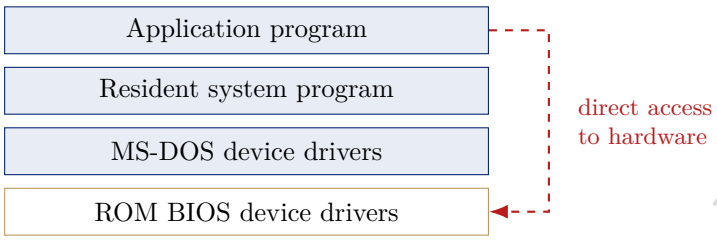


Figure 4.1: MS-DOS layer structure. The dashed path shows an application bypassing the middle levels to reach the hardware directly — cf. Silberschatz et al., Ch. 2 §2.7, pp. 70–76 [1].

4.4.2 Monolithic structure: traditional UNIX

Traditional UNIX has two parts: the **kernel** and the **system programs** (Figure 4.2). Everything between the system-call interface and the hardware interface — file system, CPU scheduling, memory management, signals, terminal handling, and device drivers — is *one* program running in kernel mode in one address space. That makes it fast, since a request from one subsystem to another is just a function call, but hard to maintain.

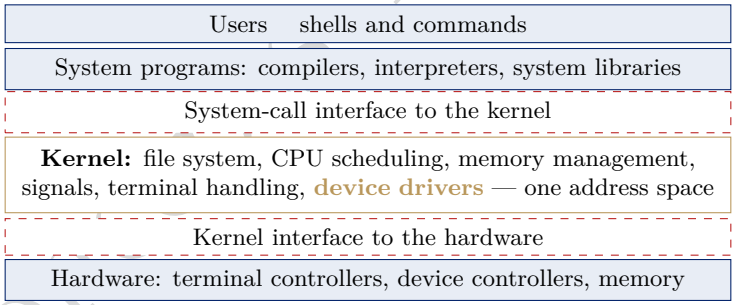


Figure 4.2: Traditional UNIX system structure: one large kernel between two interfaces — cf. Silberschatz et al., Ch. 2 §2.7, pp. 70–76 [1].

Example 4.3 (The bad driver in a monolithic kernel)**.** *Follow the buggy driver step by step.*

1. *The driver is compiled into the kernel and runs in **kernel mode** in the kernel's address space.*
2. *The rare packet arrives, and the driver dereferences* `NULL`*.*
3. *The fault happens in kernel mode, so there is no more-privileged code to contain it.*
4. *The kernel cannot tell which of its data structures the driver corrupted, so it halts (a* kernel panic*).*

Outcome: *the blast radius is the whole machine; every process, not just the network, is lost. The same speed that makes monolithic kernels attractive — no boundary between subsystems — is what lets one bug reach everything.*

Concepts That Click

4.5 The Layered Approach

Reading: Silberschatz et al., Ch. 2 §2.7, pp. 70–76; Stallings, Ch. 2 §2.3, p. 64.

In the layered approach the operating system is a stack of **layers**, from layer 0 (the hardware) to layer N (the user interface), and layer i uses only the operations of layers below i (Figure 4.3). Each layer hides its data structures from the layers above. The great advantage is that the system can be debugged bottom-up: once layer $i-1$ has been verified, any bug found while testing layer i must be in layer i.

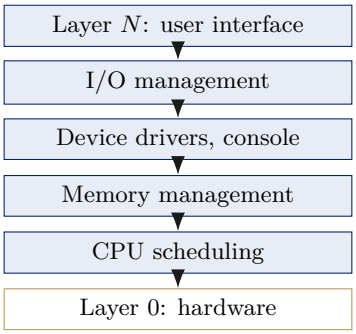


Figure 4.3: A layered operating system; each arrow is a call to the layer immediately below — cf. Silberschatz et al., Ch. 2 §2.7, pp. 70–76 [1]; Stallings, Ch. 2 §2.3, p. 64 [2].

Example 4.4 (Where does the backing-store driver go?)**.** *The rule is: if A calls B, then B must be in a* lower *layer than A.*

1. *Memory management swaps pages to disk, so it calls the backing-store driver. Therefore the* ***driver is below memory management****.*
2. *The driver waits for I/O, and the CPU should run someone else in the meantime, so the driver calls the CPU scheduler. Therefore the* ***scheduler is below the driver****.*
3. *But on a large system the scheduler's tables may not fit in memory and must themselves be swapped, so the scheduler calls the driver. Therefore the* ***driver is below the scheduler****.*

Steps 2 and 3 demand opposite orders. Choosing the layers requires careful planning, and some dependencies have no clean order at all.

The example illustrates the first cost of layering: **defining the layers is hard**, and the cycle above is typical. The second cost is that **each layer adds overhead**: a request from the top passes through every layer below it, each adding a call and possibly copying parameters. A system call that needs I/O traverses I/O → memory → scheduling → hardware, even when a direct path would do. Modern systems therefore use fewer, more functional layers, keeping the benefits without the full cost.

Example 4.5 (The bad driver, layered)**.** *Layering organizes code; it does not change privilege. The driver layer still runs in kernel mode, so the* `NULL` *dereference still crashes the whole kernel. The win from layering is finding the bug sooner, not surviving it.*

Concepts That Click

4.6 Microkernels

Reading: Silberschatz et al., Ch. 2 §2.7; Stallings, Ch. 4 §4.3, p. 179.

If kernel mode is where one bug becomes everyone's bug, the natural response is to put as little as possible in kernel mode. A **microkernel** keeps only message passing (IPC), basic memory management, and CPU scheduling in the kernel, and moves everything nonessential — file systems, drivers, networking — into **user-level servers** (Figure 4.4). Clients and servers never call each other directly: they exchange **messages** through the microkernel. Mach (developed at CMU in the mid-1980s) and QNX are the standard examples.

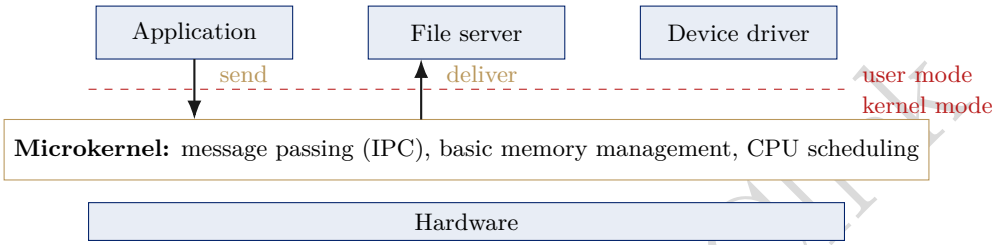


Figure 4.4: Microkernel structure. One message: the application sends to the microkernel, which delivers to the file server — cf. Silberschatz et al., Ch. 2 §2.7; Stallings, Ch. 4 §4.3, p. 179 [2].

Example 4.6 (Counting boundary crossings)**.** *The application reads a file block. Count the traps, where each trap is one user→kernel entry plus one kernel→user return, i.e. 2 crossings.*

1. ***Monolithic.*** *One* `read` *trap; the file system and driver are kernel functions. Result:* ***1 trap, 2 crossings****.*
2. ***Microkernel, file server only.*** *The application sends to the server (1 trap) and the server replies to the application (1 trap). Result:* ***2 traps, 4 crossings****.*
3. ***Microkernel, file server and driver server.*** *The messages are app→fs, fs→drv, drv→fs, fs→app. Result:* ***4 traps, 8 crossings****.*

General pattern (our counting model). *A request that passes through a chain of k servers and back sends k messages on the way out and k replies on the way back, so it costs $2k$ message traps, i.e. $4k$ crossings, versus 2 crossings for a monolithic call. With $k = 2$ that is $8/2 = 4$ times as many. Each message may also be copied between address spaces; this is the "system-function overhead" that makes microkernels slower.*

The benefits of the microkernel are that it is extensible (a new service is a new user-level server and the kernel is untouched), portable (little hardware-specific code lives in the kernel), and reliable and secure (most services run as user processes). Stallings, Ch. 4 §4.3, p. 179, lists further benefits: uniform interfaces, flexibility, and support for distributed systems. The cost is performance: messages and extra crossings for every service request.

Example 4.7 (The bad driver in a microkernel)**.** *The driver is a* user process*. The* `NULL` *dereference causes an ordinary page fault; the kernel terminates that process only, and can restart it. The blast radius is the network driver; other processes keep running.*

The history of Windows NT shows the trade-off in practice (Silberschatz et al., Ch. 2 §2.7). NT began with a layered microkernel design and later moved more services back into the kernel.

Concepts That Click

Its designers were solving the cost of the microkernel design: every service request had become messages and extra mode switches. Moving services into kernel space turns messages back into function calls, which is faster — but the price is the isolation just described: a bug in a moved service can again crash the kernel.

4.7 Modules and Hybrid Systems

Reading: Silberschatz et al., Ch. 2 §2.7, pp. 70–76; Bovet & Cesati, Ch. 1, pp. 1–34.

A modular kernel consists of a small **core kernel** plus **loadable modules** that are linked in at boot time or at run time, without recompiling the kernel. Each module has a defined interface, like a layer; any module may call any other, like a microkernel's servers; yet no messages are needed, because all modules run in kernel mode. Solaris is organized as a core kernel with seven types of loadable modules: scheduling classes, file systems, loadable system calls, executable formats, STREAMS modules, miscellaneous modules, and device and bus drivers. Linux is monolithic with loadable modules [3].

Real kernels mix structures to balance speed against isolation. **Linux** is monolithic plus loadable modules. **Mac OS X** combines a Mach microkernel with a BSD kernel environment, plus loadable kernel extensions. **Windows** is largely monolithic for performance, with microkernel-style user-mode subsystems.

Example 4.8 (The bad driver as a module)**.** *`insmod netdrv.ko` links the driver into the running kernel, in **kernel mode**. The `NULL` dereference therefore crashes the kernel, exactly as in the monolithic case. Modules buy* flexibility *(load, unload, and update without a reboot), not* isolation*.*

4.8 Virtual Machines

Reading: Silberschatz et al., Ch. 2 §2.8, p. 76.

Pushing the isolation idea one level further, we can run the entire operating system as a guest, on hardware that is itself simulated. The **virtual-machine manager** (hypervisor) gives each guest the illusion of its own processor, memory, and devices (Figure 4.5), and each guest runs a full, unmodified operating system — possibly a different one in each virtual machine. IBM's VM/370 (1972), VMware, and the Java Virtual Machine are the standard examples.

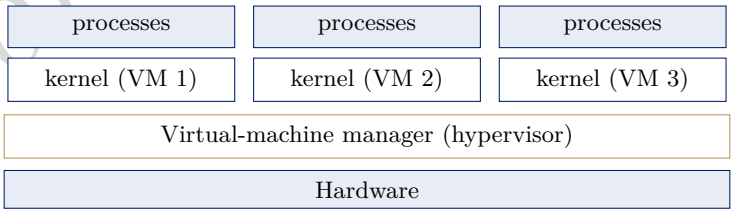


Figure 4.5: Three virtual machines on one virtual-machine manager — cf. Silberschatz et al., Ch. 2 §2.8, p. 76 [1].

Virtual machines bring isolation (each VM is protected from the others, so a virus in one cannot reach another directly), support for system development (a new kernel can be tested in a VM without disrupting normal work on the same machine), and consolidation (several lightly used servers become VMs on one physical machine). The cost is that the guest kernel itself runs in a de-privileged mode, so its privileged operations must be trapped and emulated by the manager.

Concepts That Click

Example 4.9 (The bad driver in a virtual machine)**.** *The driver runs in the* guest *kernel of VM 1. The guest panics, so all of VM 1 is lost, but the manager and VMs 2 and 3 keep running. The blast radius is one virtual machine.*

4.9 Comparison

Table 4.3 gathers the five outcomes of the running example. The pattern is that isolation is bought with boundary crossings, and every structure picks a point on that trade-off.

Structure	Driver runs in	Bug's blast radius	Speed	Main strength
Monolithic	kernel mode	whole machine	fastest (calls)	performance
Layered	kernel mode	whole machine	layer overhead	bottom-up debugging
Microkernel	user mode	the driver only	messages, 4k crossings	isolation, extensibility
Modular	kernel mode	whole machine	near monolithic	load at run time
Virtual machine	guest kernel	one VM	trap-and-emulate	isolates whole OSes

Table 4.3: One driver bug, five structures.

Example 4.10 (Choosing a structure for a flight-control computer)**.** *An aircraft flight-control computer must keep running even if its network-logging driver fails. A* **microkernel** *is the natural choice: the logging driver runs as a user-level server, so its failure cannot take down the control software. The price is that every driver request is a message — with a chain of k servers, 4k crossings instead of 2. This is why microkernels such as QNX are common in embedded and real-time systems.*

Common misconceptions

- "The shell is part of the kernel." — No: the shell is a system program running in user mode.
- "Layered means the upper layers run in user mode." — No: layering organizes code; privilege is unchanged.
- "Loadable modules isolate drivers." — No: modules run in kernel mode, and a bug crashes the kernel.
- "Microkernels are faster because the kernel is small." — No: messages add crossings; microkernels trade speed for isolation.

Summary

System programs (file management, status information, file modification, language support, program loading and execution, communications) run in user mode on top of system calls. Designers separate **mechanism** (how) from **policy** (what). A **monolithic** kernel is fast and lives in one address space, so one bug crashes all; a **layered** kernel is easy to debug but hard to order and adds overhead; a **microkernel** keeps a minimal kernel and runs services as user-level servers, paying for

Concepts That Click

isolation in message crossings; a **modular** kernel gains run-time flexibility but is still kernel mode; and a **virtual machine** isolates entire operating systems.

The next lecture turns to the unit every one of these structures manages — the **process**, and the kernel record that describes it, the **process control block**. The reading is Silberschatz et al., Ch. 3 §3.1 [1], and Stallings, Ch. 3 §3.1–3.3 [2].

For this chapter the page-cited sources are: Silberschatz et al., 8th ed., Ch. 2 §2.5 (system programs, p. 66), §2.6 (design and implementation), §2.7 (OS structure, pp. 70–76), and §2.8 (virtual machines, p. 76), with §2.5–2.8 spanning pp. 66–84; Stallings, 6th ed., Ch. 2 §2.3 (system structure, p. 64) and Ch. 4 §4.3 (microkernels, p. 179); and Bovet & Cesati, 3rd ed., Ch. 1 (kernel architecture: monolithic with loadable modules, pp. 1–34). All are page-verified through each book's `index.md` under `master_supporting_docs/CS502/supporting_books/`.

Self-Check

Exercises

1. Classify `cp`, `who`, `grep`, `gdb`, and `mail` by system-program category.
2. Why must `cd` be a shell built-in while `rm` can be a separate program?
3. For each of the following, name the mechanism and the policy: (a) the timer that preempts a running process; (b) the permission check on `open`.
4. In our counting model, a request passes through a chain of $k = 3$ servers and back. How many message traps and how many crossings does it cost, and how many times the monolithic cost is that?
5. State the blast radius of the buggy driver under a modular kernel and under a virtual machine, and explain the difference.

Solutions

1. `cp`: file management; `who`: status information; `grep`: file modification; `gdb`: programming-language support; `mail`: communications.
2. A separate program runs in a child created by `fork`; `chdir` there changes only the child's directory, and the child exits, so the shell's directory would not change. `rm` only needs to remove a file, which any process may request, so it can be a separate file that the shell loads.
3. (a) Mechanism: the timer interrupt after a set interval; policy: how long the interval is. (b) Mechanism: the kernel's check of a permission bit before `open`; policy: who is granted which bits.
4. $2k = 6$ message traps, $4k = 12$ crossings, versus 2 crossings for a monolithic call: $12/2 = 6$ times as many.
5. Modular: the module runs in kernel mode, so the `NULL` dereference crashes the kernel — the whole machine. Virtual machine: the driver runs in a guest kernel, so only that VM is lost; the manager and the other VMs keep running. Modules buy flexibility; virtual machines buy isolation.

References

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 8th ed., John Wiley and Sons — Ch. 2 §2.5 (p. 66), §2.6, §2.7 (pp. 70–76), §2.8 (p. 76). *(Page-cited; index built 2026-10-14.)*

Concepts That Click

[2] W. Stallings, *Operating Systems: Internals and Design Principles*, 6th ed., Prentice Hall — Ch. 2 §2.3 (p. 64) and Ch. 4 §4.3 (p. 179). *(Page-cited; index built 2026-10-14.)*

[3] D. P. Bovet and M. Cesati, *Understanding the Linux Kernel*, 3rd ed., O'Reilly Media, 2006 — Ch. 1 (pp. 1–34). *(Page-cited; index built 2026-10-14.)*

Concepts That Click